

Chapter 3

INTERNAL REPRESENTATION OF FILES

Contents

- -3.1 Inodes
 - 3.1.1 Definition
 - 3.1.2 Accessing
 - 3.1.3 Releasing
- -3.2 Structure of a Regular File
- -3.3 Directories
- -3.4 Conversion of a Pathname to an Inode
- -3.4 Super Block
- -3.6 Inode Assignment to a New File
- -3.7 Allocation of Disk Blocks
- -3.8 Other File Types

File System Algorithms

namei				
	alloc	free	ialloc	ifree
iget	iput			
buffer allocation algorithms				
getblk brelse bread bwrite				

Figure 1: Lower Level File system Algorithms

3.1 Inodes

- contains the information necessary for a process to access a file
- exists in a static form on disk and the kernel reads them into an in-core inode

3.1.1 Definition

- Inode consists of
 - file owner identifier
 - file type
 - file access permissions
 - file access times
 - number of links to the file
 - table of contents for the disk address of data in a file
 - file size

Cont...

- in-core copy of the inode contains
 - status of the in-core inode
 - logical device number of file system
 - inode number
 - pointers to other in-core inodes
 - reference count

3.1.2 Accessing Inodes

`iget (inode_no) //getIncoreInode`

- while (not done)
 - if (inode in inode cache)
 - if (inode locked)
 - sleep(event inode becomes unlocked)
 - continue
 - if (inode on inode free list)
 - remove from free list
 - return locked inode
 - if (no inode on free list) return error
 - remove new inode from free list
 - set inode number
 - remove inode from old hash queue and place on new one
 - read inode from disk
 - set reference count 1
 - return locked inode

3.1.3 Releasing Inodes

iput (inode_no) //releaseIncoreInode

- lock inode if not locked
- decrement inode refernece count
- if (refernce count==0)
 - if (inode link==0)
 - free disk block
 - set file type to 0
 - free inode
 - if (file accessed or inode changed or file changed)
 - update disk inode
 - put inode on free list
- Release inode lock

3.2 Structure of a regular file

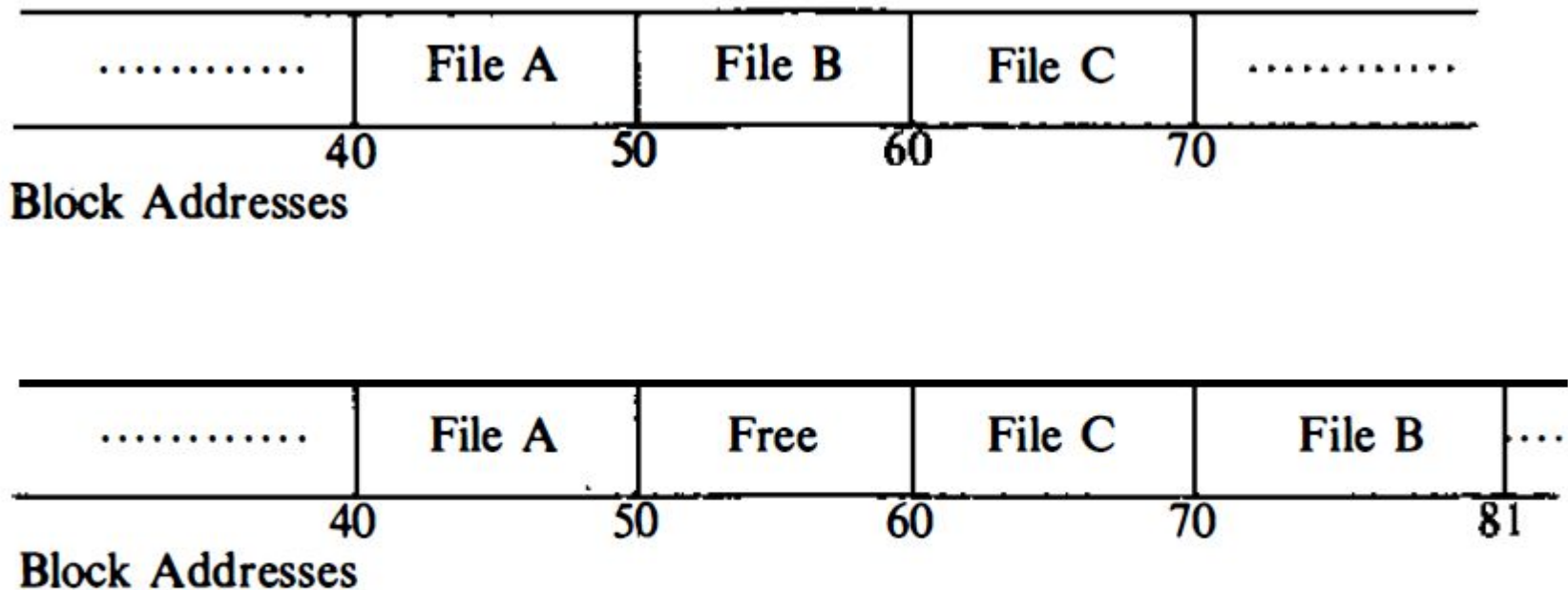


Figure 4.5. Allocation of Contiguous Files and Fragmentation of Free Space

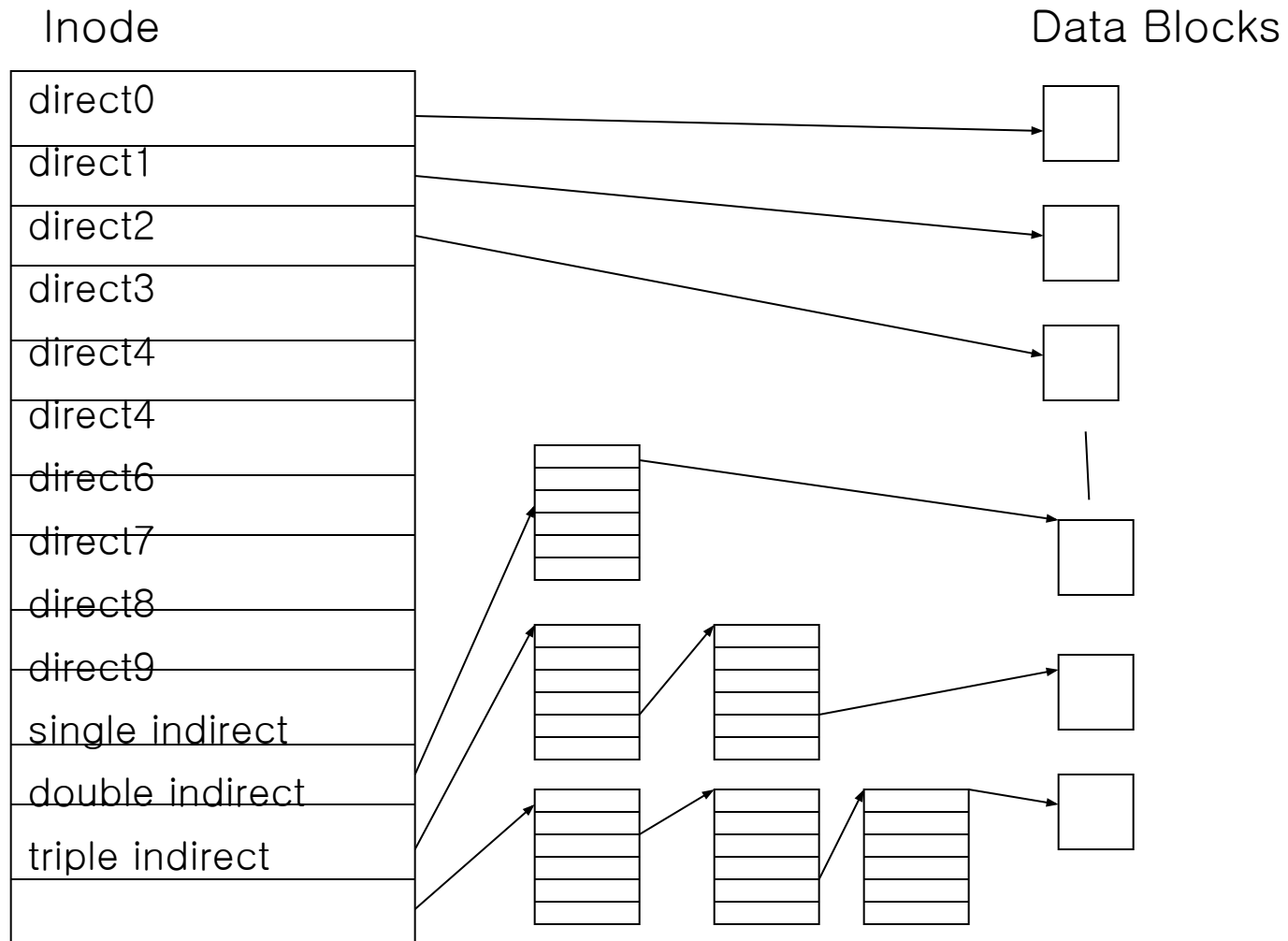


Figure 6 Direct and Indirect Blocks in Inode

Cont....

Suppose System V UNIX

Assume that a logical on the file system holds 1K bytes and that a block number is addressable by a 32 bit integer, then a block can hold up to 246 block numbers

10 direct blocks with 1K bytes each=10K bytes

1 indirect block with 246 direct blocks= $1K * 246 = 246K$ bytes

1 double indirect block with 246 indirect blocks=
 $246K * 246 = 64M$ bytes

1 triple indirect block with 246 double indirect blocks=
 $64M * 246 = 16G$ bytes

```
bmap algorithm
  input: inode/byte offset
  output: block #, offset
          bytes of i/o in block
          readahead block #

  .calc logical block # from byte offset
  .calc starting byte in block
  .calc # of bytes to copy to user
  .check if read-ahead applicable, mark inode
  .determine level of indirection
  .while not at necessary level
    .calc index into inode or indirect block form logical
    block number in file
    .get disk block# from inode or indirect block
    .release buffer form previous disk read if any
    .if no more indirection, return block #
    .read indirect disk block
      bread
    .adjust logical blkno acc. to level of
    indirection
```

Figure 8. Conversion of Byte Offset to Block Number in File System

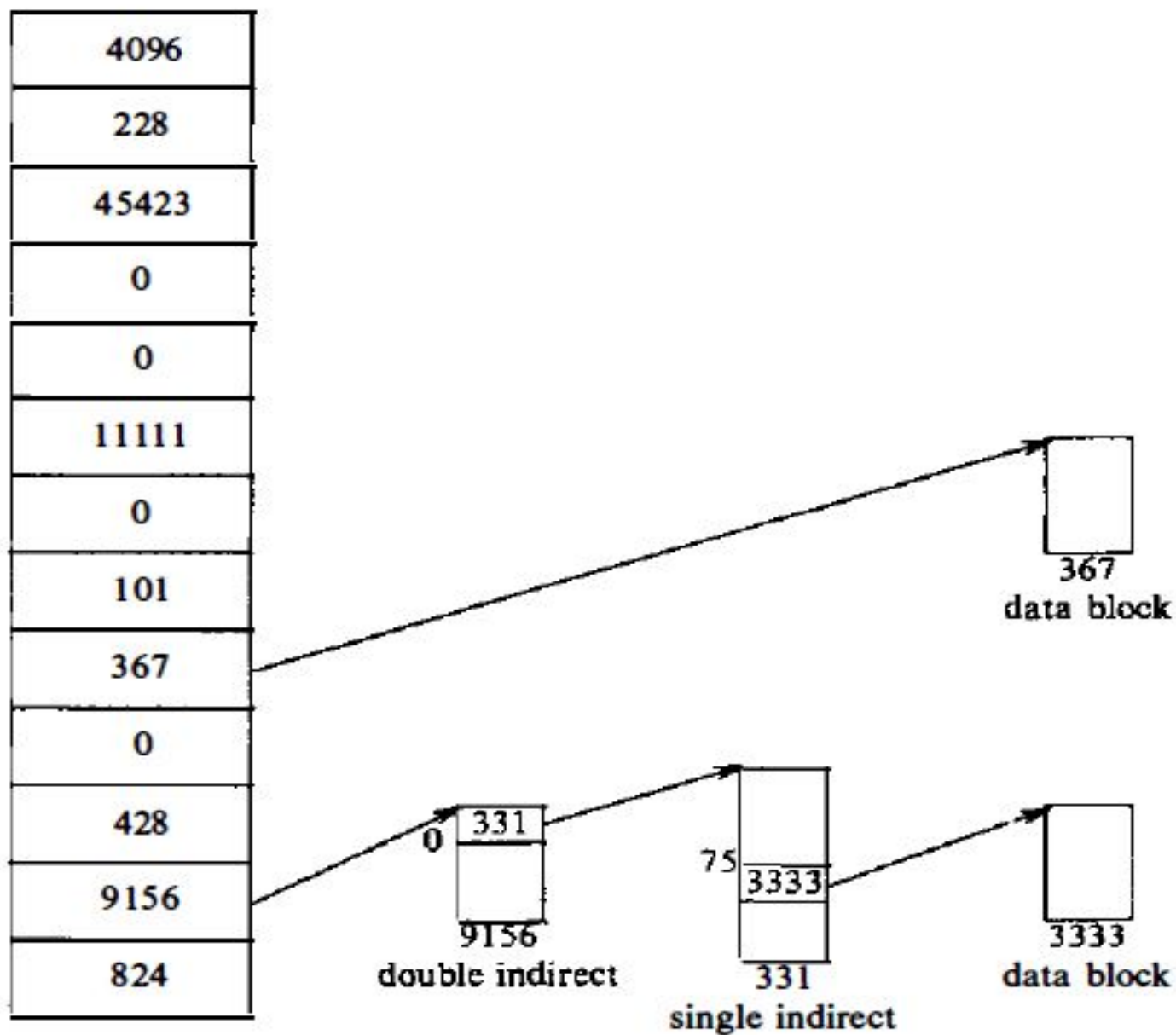


Figure 4.9. Block Layout of a Sample File and its Inode

3.3 Directories

- .file
- .array of directory records
- .inode, filename pairs
 - struct directory {
 - short inode;
 - char name[14];
 - };
- .DOT and DOTDOT are a requirement, i.e., self and parent.
- .kernel ONLY writes to directories; processes can read them but can't write to them.
- .creat/mknod/link/unlink modify.
- .empty slots get a 0 inode to mark as empty.
- .directory files never shrink
- .BSD system uses variable length filenames.
This means directory structure is variable length record.
- .readdir routines, opendir, closedir, readdir commonly used to access directory so as to hide true directory structure. POSIX standard.

Byte Offset in Directory	Inode Number (2 bytes)	File Names
0	83	
16	2	..
32	1798	init
48	1276	fsck
64	85	clri
80	1268	motd
96	1799	mount
112	88	mknod
128	2114	passwd
144	1717	umount
160	1851	checklist
176	92	fsdb1 b
192	84	config
208	1432	getty
224	0	crash
240	95	mkfs
256	188	inittab

Figure 4.10. Directory Layout for /etc

3.4 Path conversion to an inode

- if (path name starts with root)
 - working inode= root inode
- else
 - working inode= current directory inode
- while (there is more path name)
 - read next component from input
 - read directory content
 - if (component matches an entry in directory)
 - get inode number for matched component
 - release working inode
 - working inode=inode of matched component
 - else
 - return no inode
- return (working inode)

3.4 Conversion of a path name to an inode

- algorithm namei
 - If path name starts from root, then the kernel assigns root inode(iget) to working inode
 - Otherwise, the kernel assigns current directory inode to working inode
 - While there is more path name, the kernel reads next path name component from input, and verifies that working inode is of directory, access permissions OK
 - If working inode is of root and component is '..', then the kernel checks whether there is more path name or not
 - Otherwise the kernel reads directory by repeated use of bmap,bread,brelse

3.4 Super block

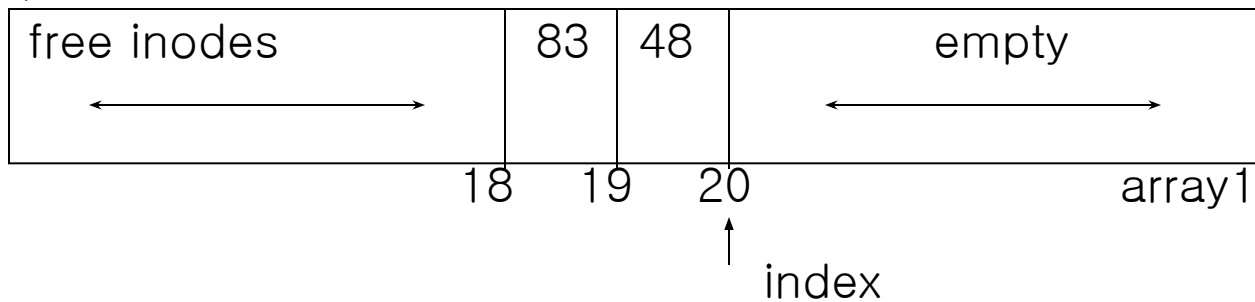
- File System

boot block	super block	inode list	data blocks
------------	-------------	------------	-------------

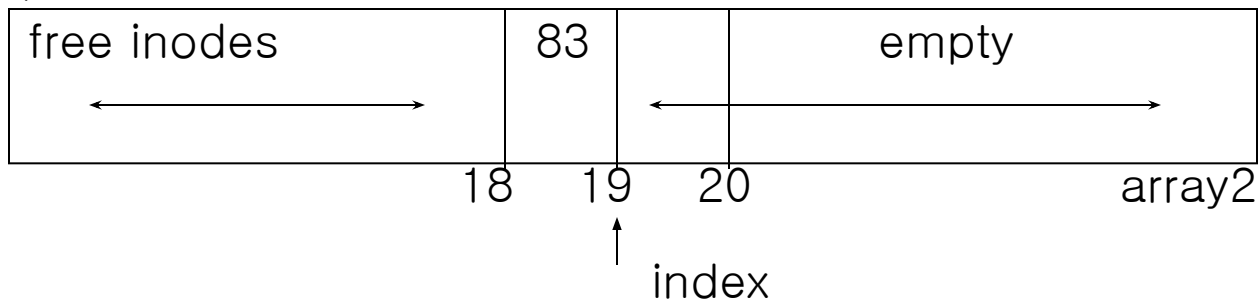
- consists of
 - the size of the file system
 - the number of free blocks in the file system
 - a list of free blocks available on the file system
 - the index of the next free block in the free block list
 - the size of the inode list
 - the number of free inodes in the file system
 - a list of free inodes in the file system
 - the index of the next free inode in the free inode list
 - lock fields for the free block and free inode lists
 - a flag indicating that the super block has been modified

3.6 Inode assignment to a new file

Super Block Free Inode List

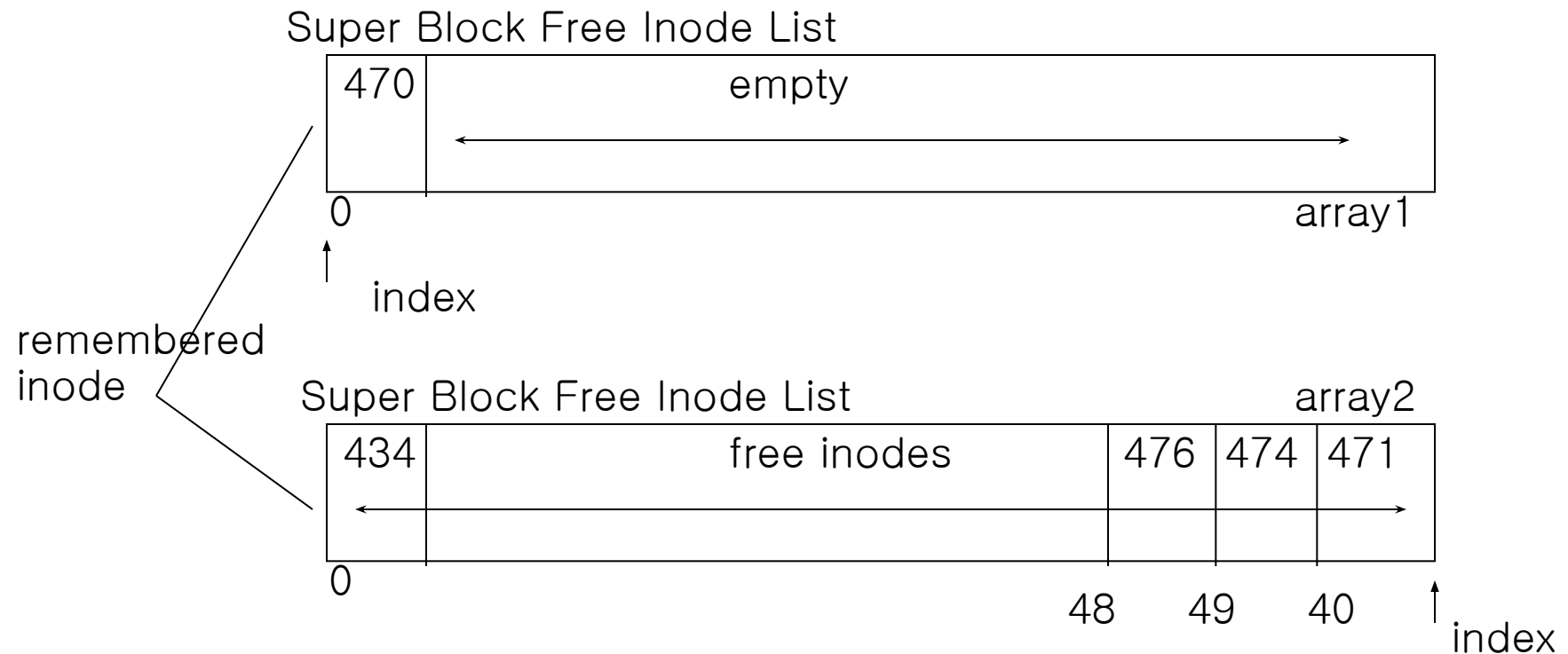


Super Block Free Inode List



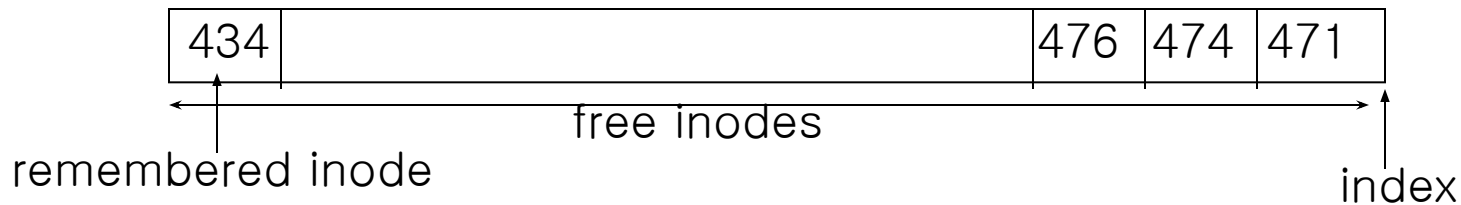
Assigning Free Inode from Middle of List

3.6 Inode assignment to a new file

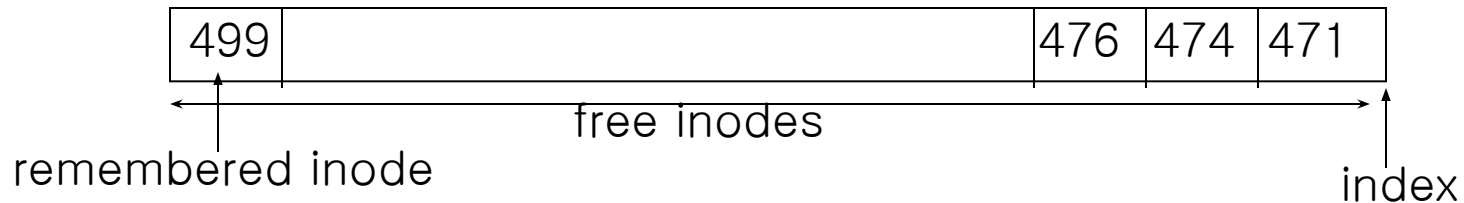


Assigning Free Inode – Super Block List Empty

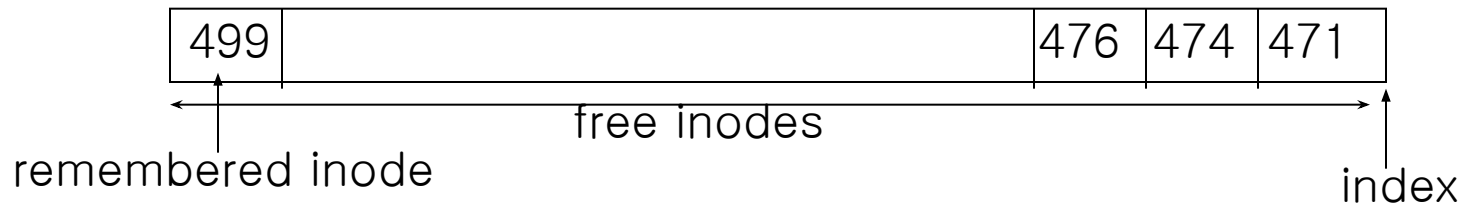
3.6 Inode assignment to a new file



Original Super Block List of Free Inodes



Free Inode 499



Free Inode 601

3.6 Inode assignment to a new file

- Locked inode illoc()
 - while (not done)
 - If (super block locked)
 - Sleep (event super block becomes free)
 - Continue
 - If (inode list in super block empty)
 - Lock super block
 - Get remember inode for free inode search
 - Search until super block full or no more free inode
 - Unlock super block and wake up (event super block free)\
 - If no free inode found on disk return error
 - Set remmbered inode for next free inode search
 - Get inode number from super block inode list
 - Get inode
 - Write inode to disk
 - Decrement free inode count
 - Return inode

Freeing inode

- `ifree(inode_no)`
 - Increment free inode count
 - If super block locked return
 - If (inode list full) //at super block
 - if (inode number < remembered inode)
 - Set remembered inode as input inode
 - Else
 - Store inode number in inode list
 - return

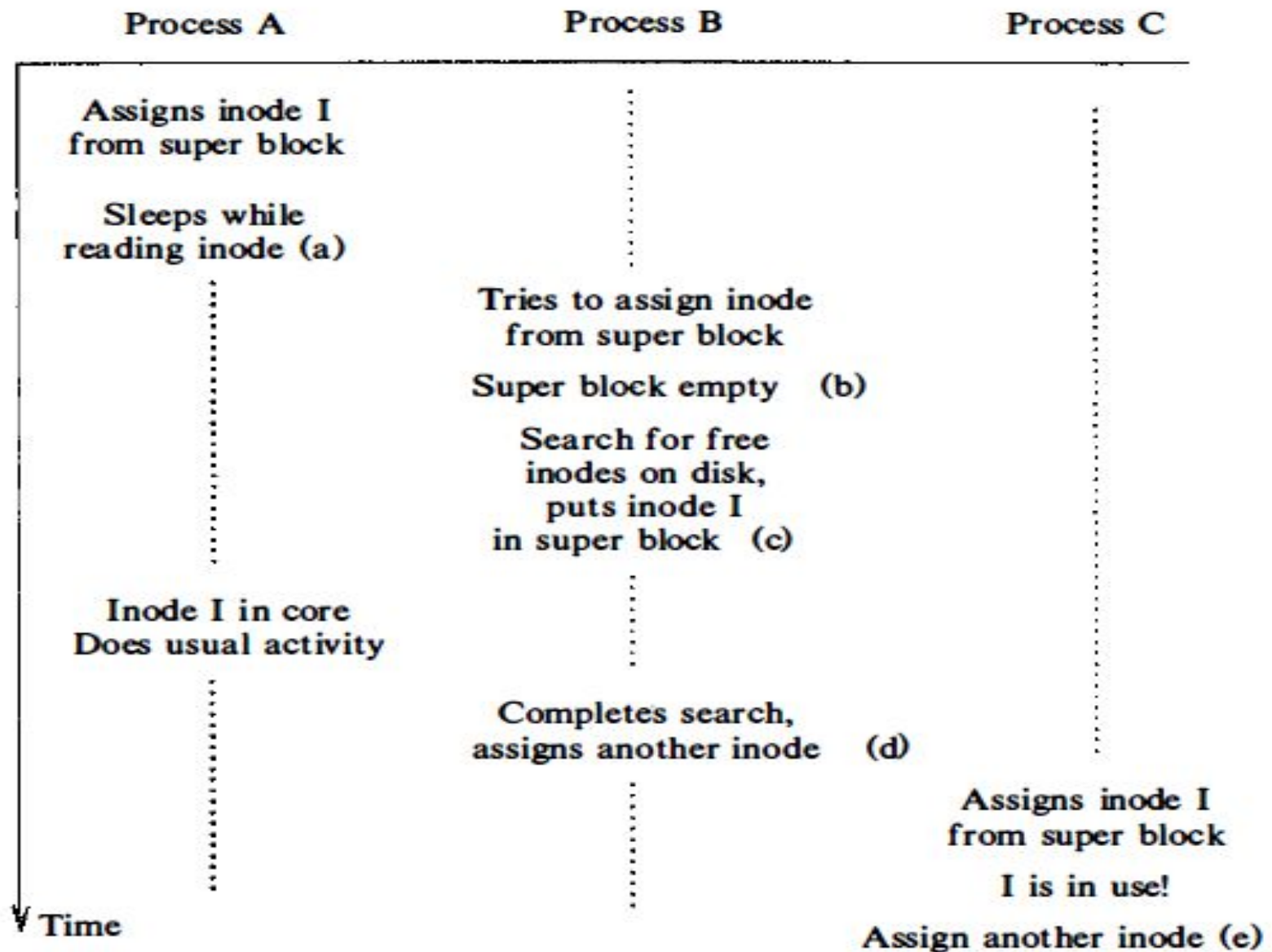


Figure 4.16. Race Condition in Assigning Inodes

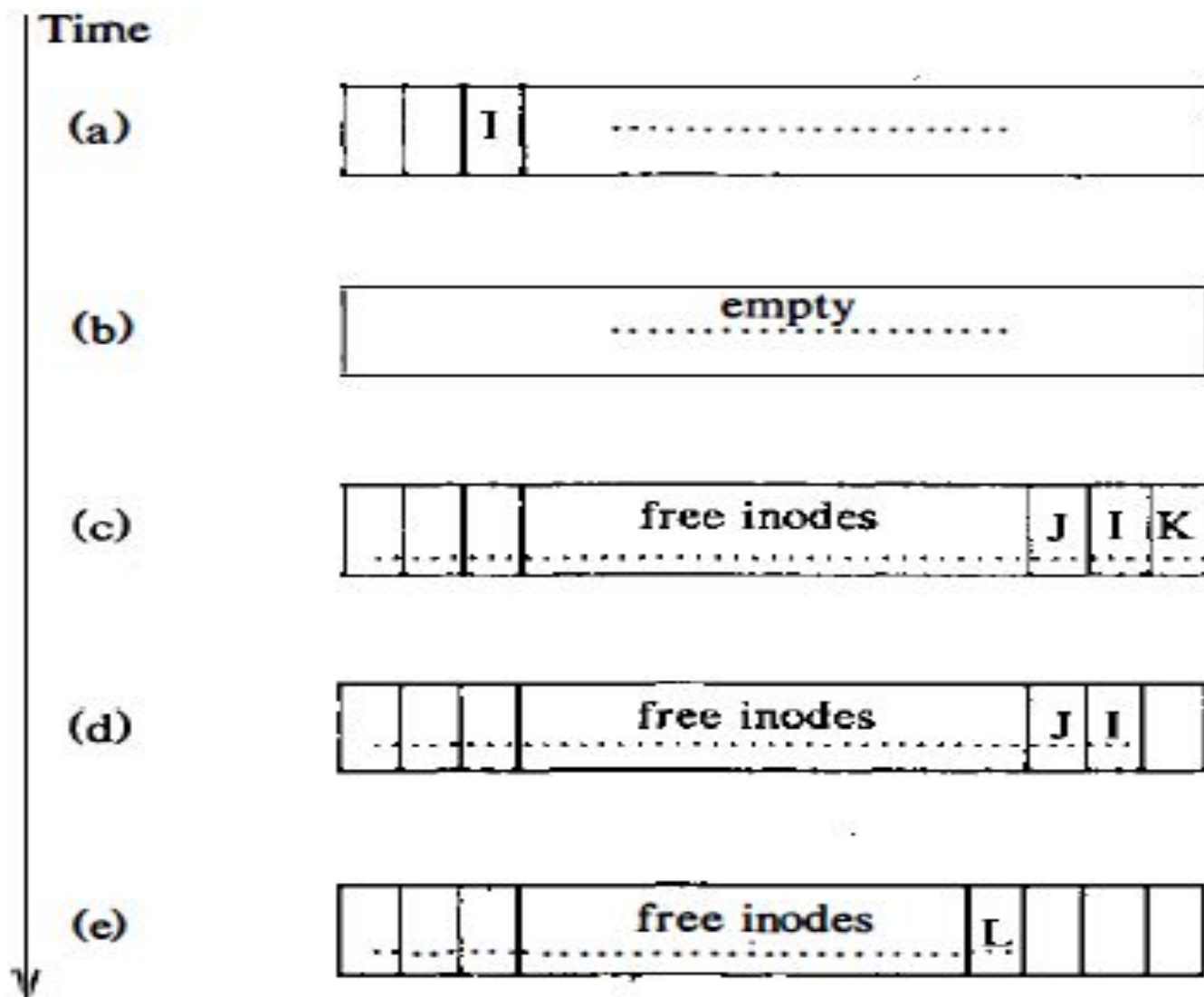
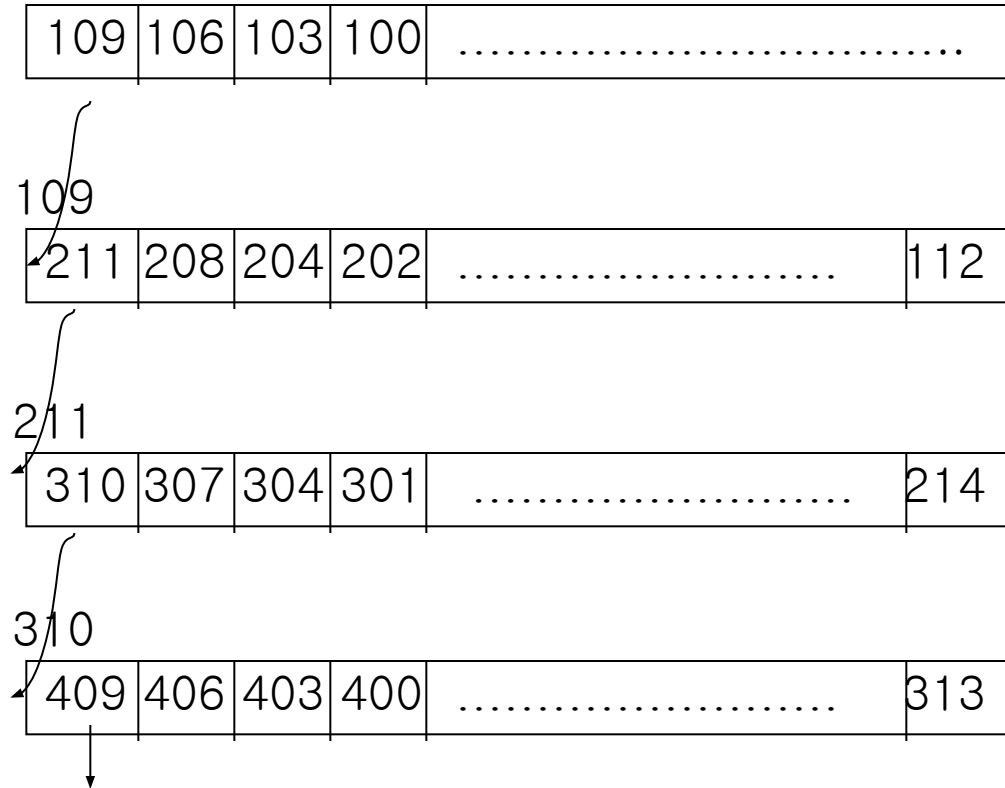


Figure 4.17. Race Condition in Assigning Inodes (continued)

3.7 Allocation of disk blocks

- when process writes data, kernel must allocate free disk blocks.
- free disk blocks rooted in SB, and link exists in SB to linked list of free blocks. i.e., SB is cache for disk blocks. SB is kept in core (mount table)
- Last entry in SB free block cache is block number for block that contains list of free blocks.
- Last entry in that block is block number for block that ... etc.
- mkfs utility builds SB/inodes/free block list to make logical Unix file system.
- no attempt made to sort blocks on free list.
- alloc() allocates a disk block
- free() frees a disk block

Cont...



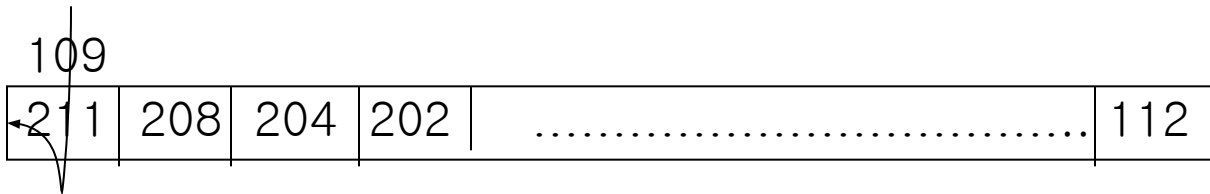
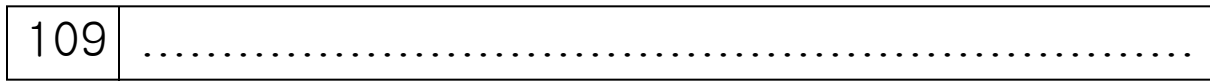
linked list of free disk block number

Cont...

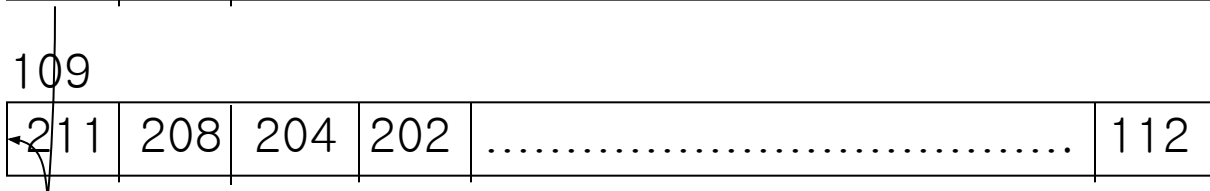
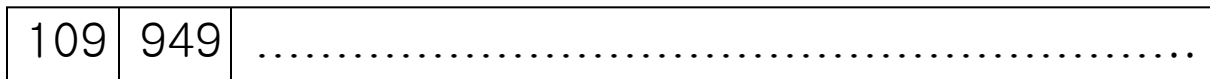
- algorithm alloc
 - The kernel wants to allocate a block from a file system
 - it allocates the next available block in the super block list
 - Once allocated , the block cannot be reallocated until it becomes free
 - If the allocated block is the last block , the kernel treats it as a pointer to a block that contains a list of free blocks
 - The kernel locks super block, reads block just taken from free list, copies block numbers in block into super block, releases block buffer, and unlocks super block
 - Otherwise,
 - The kernel gets buffer for block removed from super block list , zero buffer contents, decrements total count of free blocks, and marks super block modified

Cont...

super block list



original configuration



After freeing block number 949

3.8 Other File Types

Pipe

- *fifo* (first-in-first-out)
- its data is transient
 - Once data is read from a pipe, it cannot be read again
 - The data is read in the order that it was written to the pipe, no deviation from that order
- using only direct block

Special File (including block device, character device)

- Specifying devices
 - The inode does not reference any data
- The inode contains the major and minor device number
- major number
 - a device type such as terminal or disk
- minor number
 - the unit number of the device

3.9

SYSTEM CALLS FOR THE FILE SYSTEM

Contents

- 1 Open
- 2 Read
- 3 Write
- 4 File and record locking
- 5 Adjusting the position of File i/o, seek
- 6 Close
- 7 File Creation
- 8 Creation of special files
- 9 chdir/chroot
- 10 chown/chmod
- 11 stat/fstat
- 12 pipes
- 13 dup
- 14 mount/umount
- 15 link
- 16 unlink
- 17 file system abstractions
- 18 file system maintenance

Introduction

- Goals
 - System calls for the file System and Algorithm
 - Understanding System call's internal procedure
- What We deals ? System Call
 - Accessing existing file
 - open, read, write, lseek, and close
 - Creating new files
 - creat, and mknod
 - Manipulating the inode or maneuvering through the file System
 - chdir, chroot, chwon, chmod, stat, and fstat

Introduction (cont.)

- Advanced system call
 - implementation of pipes : pipe , dup
 - extending file system tree : mount , umount
 - change the structure of the file system hierarchy : link and unlink
- What We deals ? Three kernel data structures
 - file table
 - user file descriptor table
 - mount table

Introduction (cont.)

File System Calls

Return File Desc	Use of namei		Assign inodes	File Attributes	File I/O	File Sys Structure	Tree Manipulation
open creat dup pipe close	open creat chdir chroot chown n chmo d	stat link unlink mkno d mount umount	creat mkno d link unlink	chown chmod stat	read write lseek	mount umount	chdir chown
Lower Level File System Algorithms							
namei			lalloc ifree	alloc free bmap			
iget iput							
Buffer allocation algorithms							
getblk brelse bread breada bwrite							

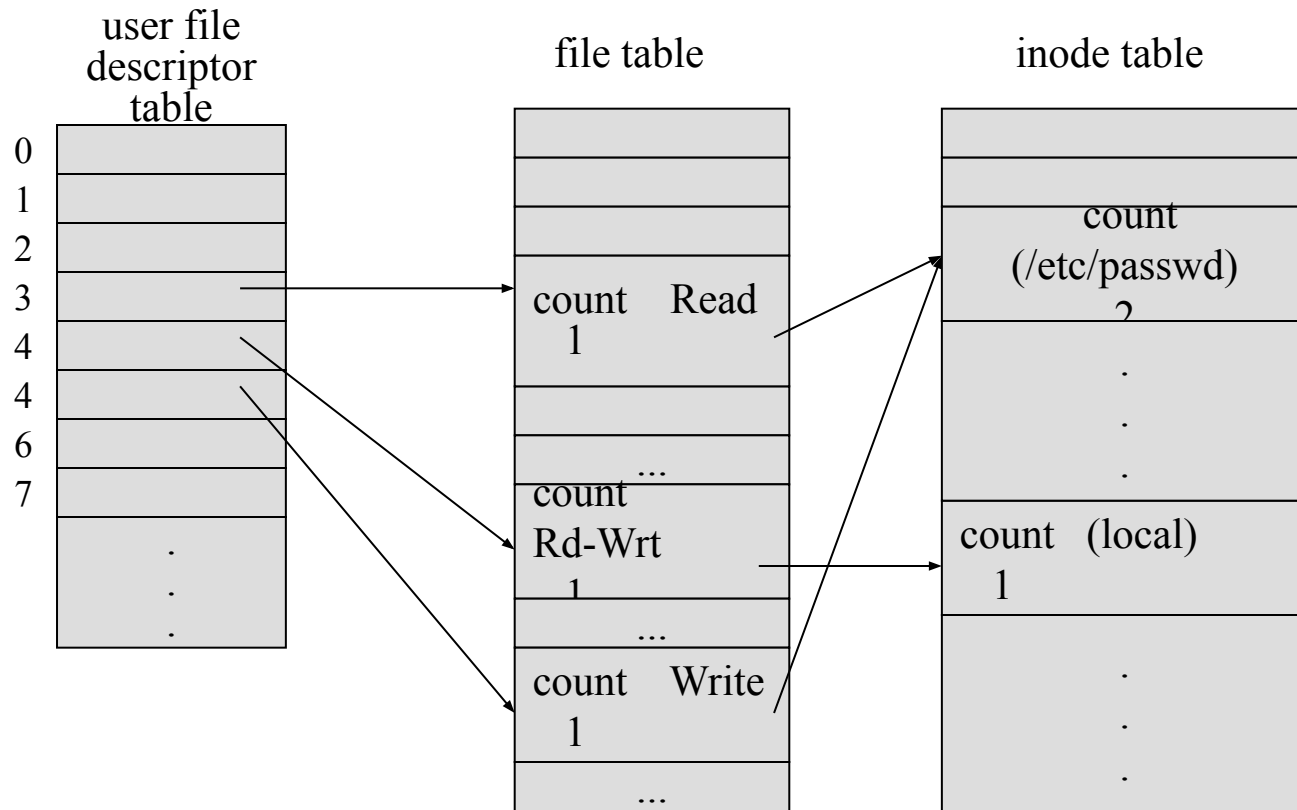
File System Calls and Relation to Other Algorithms

1 OPEN

- Syntax
 - `fd = open(pathname, flags, modes);`
 - `pathname`: file name
 - `flags` : type of open
 - `modes` : file permission if file is being created
 - `fd` : File Descriptor
- File Descriptor
 - Used in file operation (reading, writing, seeking, duplicating the file descriptor, setting file I/O parameters, determining file status, and closing file)

OPEN Algorithm

```
{
  convert file name to inode(algorithm namei);
  if(file does not exist or not permitted access)
    return (error);
  allocate file table entry for inode, initialize count, offset;
  allocate user file descriptor entry, set pointer to file table entry;
  if(type of open specifies truncate file)
    free all file blocks (algorithm free);
  unlock(inode)      /* locked above in namei */
  return(user file descriptor);
}
```



Data Structure after Open

```
fd1 = open("/etc/passwd", O_RDONLY);
fd2 = open("local", O_RDWR);
fd3 = open("etc/passwd", O_WRONLY);
```

- Note
 - Why Does not User File Table entry contain file offset for the position of the next I/O operation and point directly to the in-core inode entry for the File ?
 - Thompson said
 - To share of the offset pointer between several user file descriptors (dup and fork)

2 READ

- syntax
 - `number = read(fd, buffer, count)`
 - `fd` : file descriptor returned by *open*
 - `buffer` : address of structure in the user process that will contain the read data
 - `count` : number of bytes the user wants to read
 - `number` : number of bytes actually read

READ Algorithm

```
get file table entry from user file descriptor;
check file accessibility;
set parameters in u area for user address, byte count, I/O to user;
get inode from file table;
lock inode;
set byte offset in u area from file table offset
while(count not satisfied) {
    convert file offset to disk block (algorithm bmap);
    calculate offset into block, number of bytes to read;
    if(number of bytes to read is 0)
        /* trying to read end of file */
    read block(algorithm breada if with read-ahead, algorithm bread otherwise);
    copy data from system buffer to user address;
    update u area fields for byte offset,read count,address to write into user space;
    release buffer;    /* locked in bread */
}
unlock inode;
update file table offset for next read;
return(total number of bytes read);
```

- I/O Parameters in U Area
 - Parameters
 - mode : indicates read or write
 - count : count of bytes to read or write
 - offset : byte offset in file
 - address : target address to copy data, in user or kernel memory
 - flag : indicates if address is in user or kernel memory
 - Why in U Area
 - to eliminate the need to pass them as function parameters.

- Why (Multiple of) Disk Block to I/O
 - To avoid an extra iteration in the read algorithm loop
 - expense of accessing the inode to find the correct block number for the data and competing with other processes for access to the buffer pool.

READ Example

- Sample Source code

```
#include<fcntl.h>
Main() {
    int fd;
    char lilbuf[20], bigbuf[1024];
    fd = open("etc/passwd", O_RDONLY);
    read(fd,lilbuf, 20);
    read(fd, bigbuf, 1024);
    read(fd, lilbuf, 20);
}
```

- Sample Analysis

- First read call
- Second read call

- What is look-ahead?
- What is the problem within this Source Program

Process A

```
#include<fcntl.h>
main() {
    int fd;
    char buf[412];
    fd = open("/etc/passwd",O_RDONLY);
    read(fd, buf, sizeof(buf));
    read(fd, buf, sizeof(buf));
}
```

Process B

```
#include<fcntl.h>
main() {
    int fd;
    char buf[412];
    fd =
    open("/etc/passwd",O_WRONLY);
    write(fd, buf, sizeof(buf));
    write(fd, buf, sizeof(buf));
}
```

3 WRITE

- Syntax
 - `number = write(fd, buffer, count);`
- algorithm
 - same as read
 - however if the file does not contain a block that correspond to the byte offset to be written, the kernel allocates a new block using algorithm *alloc* and assigns the block number to the correct position in the inode's table of contents.
 - through internal loop, write one block to disk during each iteration.
- delayed write

4 FILE AND RECORD LOCKING

- File Locking
 - Capability to prevent other processes from reading or writing any part of an entire file,
- Record Locking
 - Capability to prevent other processes from reading or writing particular record

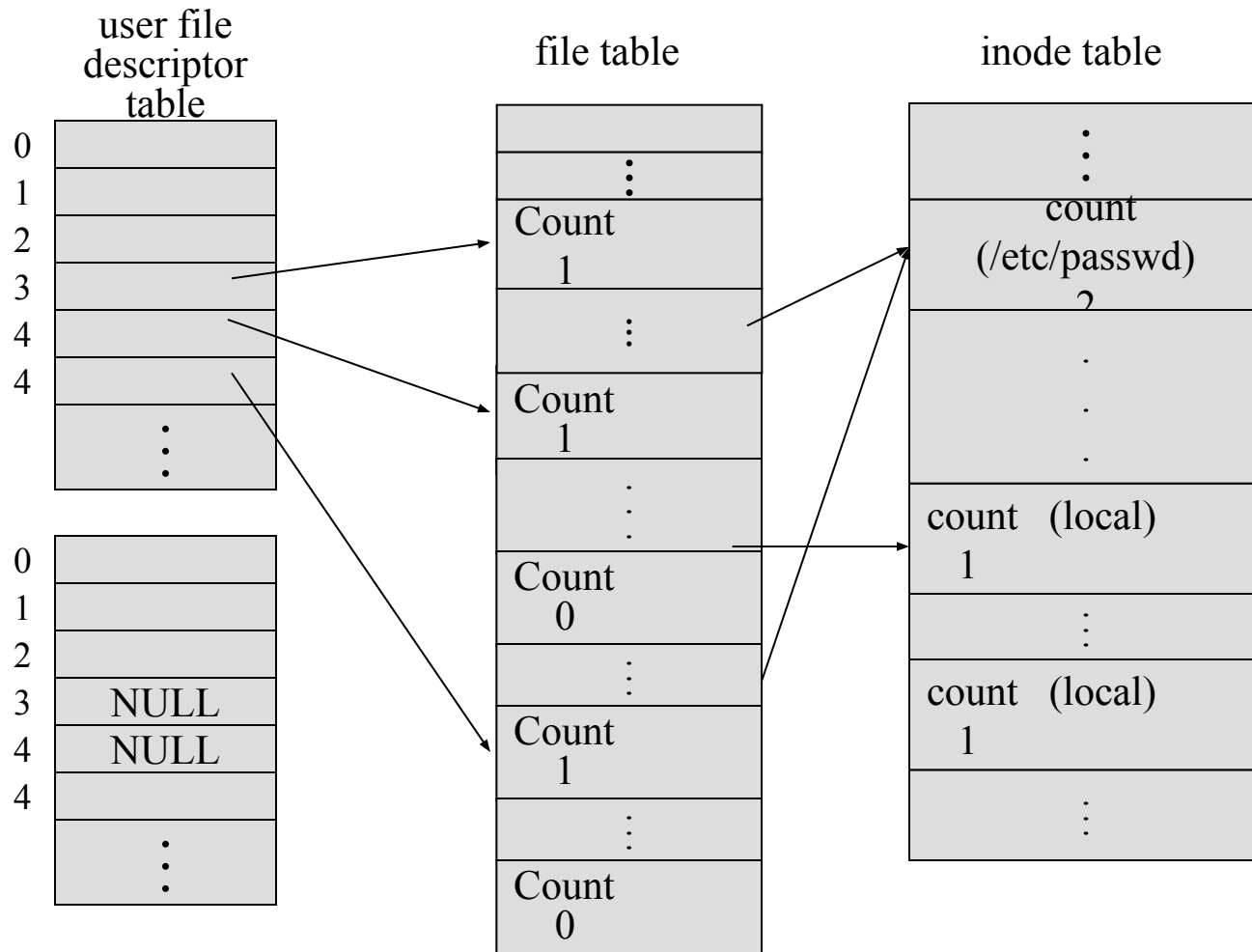
5 ADJUSTING THE POSITION OF FILE I/O

LSEEK

- Goal
 - To position the I/O and allow random access to a file
- Syntax
 - `position = lseek(fd, offset, reference);`
 - Reference :-> set the beginning of offset position
 - beginning of the file(0)
 - current position of read/write offset(1)
 - end of the file(2)
 - position: byte offset where the next read or write will start
- Function
 - kernel simply adjusts the offset value in the file table

6 CLOSE

- Goal
 - Close an open file
- Syntax
 - `close(fd);`



Tables after Closing a file

7 FILE CREATION

- Goal
 - Create a New File
- Syntax
 - `fd = creat(pathname, modes);`
- Function
 - If File does not exist
 - kernel create new file with specified name and permission mode
 - If File exists
 - truncates the file(release all existing data blocks and sets the file size to 0) subject to suitable file access permissions.

CREATE Algorithm

```
get inode for file name (algorithm namei);
if(file already exists)
{
    if(not permitted access)
    {
        release inode (algorithm iput);
        return(error);
    }
}
else    /* file does not exist yet */
{
    assign free inode from file system(algorithm ialloc);
    create new directory entry in parent directory; include new file name      and      newly
    assigned inode number;
}
allocate file table entry for inode, initialize count;
if(file did exist at time of create)
    free all file blocks (algorithm free);
unlock(inode);
return(user file descriptor)
```

8 CREATE OF SPECIAL FILES - mknod

- Goal
 - creates special files
 - named pipes
 - device files
 - directories
- Syntax
 - `mknod(pathname, type and permissions, dev)`
 - `pathname`: name of the node to be created
 - `type and permissions` : node type and access permission
 - `dev`: specifies the major and minor device numbers for block and character special files
- Function
 - allocates inode for the file

- ## Algorithm

```
if(new node not named pipe and user not super user)
    return(error);
get inode of parent of new node ( algorithm namei );
if(new node already exists)
{
    release parent inode ( algorithm iput );
    return(error);
}
```

```
assign free inode from file system for new node (algorithm ialloc);
create new directory entry in parent directory
    : include new node name and newly assigned inode nubmer;
release parent directory inode (algorithm iput);
```

```
if(new node is block or character special file)
    write major, minor numbers into inode structure;
```

```
release new node inode (algorithm iput);
```

9 CHANGE DIRECTORY AND CHANGE ROOT

9.1 CHANGE DIRECTORY

- Goal
 - Change the current directory of a process
- Syntax
 - `chdir(pathname);`
 - `pathname` : directory that becomes the new current directory of the process

- Algorithm

```
get inode for new directory name (algorithm namei);  
if(inode not that of directory or process not permitted access to file)  
{  
    release inode (algorithm input);  
    return(error);  
}  
unlock inode;  
release "old" current directory inode (algorithm input);  
replace new inode into current directory slot in u area;
```


9.2 CHANGE ROOT

- Goal
 - Change the notion of the file system root
- Syntax
 - `chroot(pathname);`
 - `pathname`: Directory that the kernel subsequently treats as the process's root directory.
- Algorithm
 - Same with `chdir`
- Notes
 - The new inode is the logical root of the file system for the process, so start from `root("/")` start from this inode

10 CHANGE OWNER AND CHANGE MODE

- Goal
 - Change the owner or mode of a file
- Syntax
 - `chown(pathname, owner, group);`
 - `chmod(pathname, mode);`
- Function
 - Chown
 - Convert the file name to inode(using algorithm namei)
 - Assign the new owner and group
 - Clear the set user and set group flags
 - Release the inode via algorithm iput
 - Chmod

11 STAT AND FSTAT

- Goal
 - To query the status of files, returning information such as the file type, file owner, access permissions, file size , number of links, inode number, and file access times.
- Syntax
 - `stat(pathname, statbuffer);`
 - `fstat(fd, statbuffer);`
- Function
 - simply write the fields of the inode into `statbuffer`(which is the address of a data structure in the user process)

12 PIPES

- Goal
 - Transfer of data between processes in FIFO
 - Synchronization of process execution
- Two Kind of Pipes
 - Named Pipe
 - by *open* system call
 - All processes can access
 - Unnamed Pipe
 - By *pipe* system call
 - Only related processes can share access
- Usage
 - Use the regular system calls for files
 - *read, write, close*

12.1 PIPES – The Pipe System Call

- Syntax
 - Pipe(fdptr)
 - fdptr : pointer to an integer array that will contain the two file descriptor for reading and writing the pipe
- Function
 - Assign inode
 - Allocate a pair of user file descriptor and corresponding file table entries for the pipe

PIPES – The Pipe System Call (cont.)

- Algorithm (for unnamed pipe)

assign new inode from pipe device(algorithm ialloc);

allocate file table entry for reading and another for writing;

initialize file table entries to point to new inode;

allocate user file descriptor for reading, another for writing, initialize to point to respective file table entries;

set inode reference count to 2;

initialize count of inode readers, writers to 1;

12.2 named pipes

- overall similar to unnamed except for setup/teardown/open
- mknod to creat
- open to open
- unlink to get rid of
- permits unrelated processes to communicate
- on open kernel counts # of processes open for r/w in inode
- if process opens pipe to read and no writer or vice versa will block until other party shows up. (no delay option can override)

12.3 reading/writing pipes

- process a writes on one end
- process b reads on the other
- fifo (circular q of buffers)
- pipes use only direct blocks, not indirect blocks
- kernel manipulates buffers as if circular queue.

12.3 reading/writing pipes (cont....)

synchronization on read/write, 4 cases

case 1: process writes, room in queue

- .on write pipe grows
- .kernel will wrap to offset 0 if needed
- .inode offset updated at end of write
- .readers woken up

case 2: process reads, enough data to satisfy

- .kernel decrements size acc. to number of bytes read
- .offset may wrap past beginning
- .all sleeping writers issued wakeup

12.3 reading/writing pipes (cont....)

case 3: process reads more data than exists

- .will read all there is and not block
- .if empty, will sleep until a writer shows up (unless no delay)
- .similar to read/write on terminal device

case 4: process writes too much data

- .sleeps until data drains

Means >1 proc writing, data could not be contiguous

12.4 closing pipes

- kernel decrements #Q pipe readers/writers acc. to the type of the fd.
 - if count of writers is 0, and there are processes waiting to read, the kernel wakes them up and read returns 0.
 - if count of readers is 0, and processes are asleep and trying to write or just trying to write, get SIGPIPE - signal, writing on broken pipe
EPIPE returned on write
 - although it is possible to get new readers/writers on FIFO, still treated the same way as unnamed pipes on close
 - if no reader/writers, kernel frees data blocks and adjust inode to indicate it is empty (FIFO).
- .if unnamed pipe, inode is deleted.

13 Dup

`newfd = dup(fd);`

copies fd into first free slot of the user file d table.

- file type does not matter

u_fd -----> File

u_fd -----^

- dup2

`dup2(oldfd, newfd)`

oldfd - the one we want to dup

newfd - the slot we want to use

newfd is closed if open

- useful in shell e.g.,

redirect output (1 == STDOUT)

`fd = creat(outfile)`

`close(1); dup(fd); close(fd);`

14 MOUNTING FILE SYSTEMS

- When UNIX is started, the directory hierarchy corresponds to the file system located on a single disk called the root device.
- The mount utility allows a super-user to splice the root directory of a file system into the existing directory hierarchy.
- File systems created on other devices can be attached to the original directory hierarchy using the mount mechanism.

Mounting File System

- Syntax

- Mount (special pathname, directory pathname, options);

- special pathname : the disk section containing the file system to be mounted
 - directory pathname : in the existing hierarchy where the file system will be mounted (called the mounted point)
 - option : ‘read-only’

- Mount

- connects the file system in a specified section of a disk to the existing file system hierarchy

- Mount table

- device number, Pointer to a buffer, Pointer to the root inode, Pointer to the inode of the directory

MOUNT(CONT')

- When mount is established, users are unaware of crossing mount points.
- File system may be detached from the main hierarchy using the umount utility.
- Links do not work across mounts (System V)
- Example:
 - `$ mount /dev/floppy /mnt`
 - `$ umount /mnt`

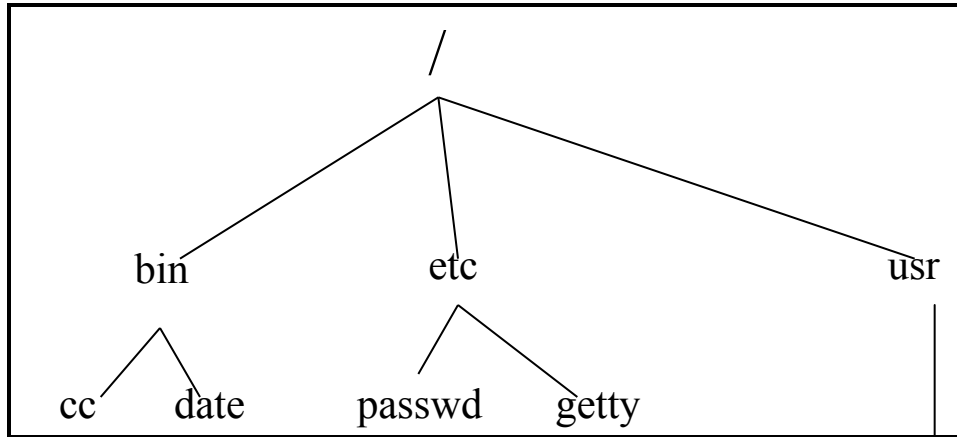
Mount (cont')

- Kernel maintains a system-wide data structure called the mount table that allows multiple file systems to be accessed via a single directory hierarchy.
- `mount( )` and `umount( )` system calls modify table, in the following manner:

MOUNT (CONT')

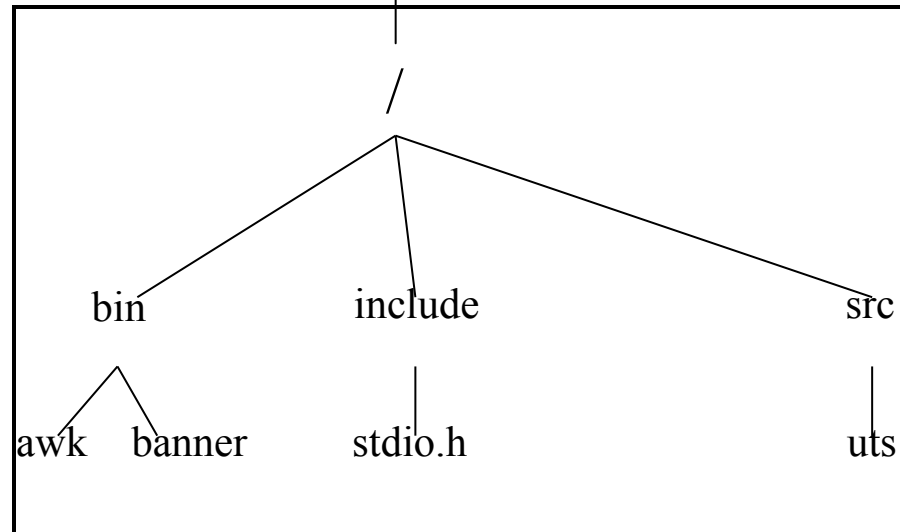
- with `mount( )`, an entry is added with:
 - device number containing file system
 - a pointer to the root inode of the newly mounted file system
 - a pointer to the inode of the mount point
 - a pointer to the filesystem-specific mount data structure of the newly mounted file system.

File System Tree Before and After Mount



Root file system

```
mount("/dev/dsk1", "/usr", 0)
```



/dev/dsk1
file system

Crossing Mount Points in File Path Name

- Crossing from the mounted-on file system to the mounted file system (root -> leaf node)
- Crossing from the mounted file system to the mounted-on file system
- Example
 - % mount /dev/disk1 /usr
 - % cd /usr/src/uts 위 -> 아래
 - 76 % cd ../../.. 아래 -> 위

Umount ()

- With umount() several checks are made in the kernel:
 - checks that there are no open files in the file system to be unmounted
 - flushes the superblock and buffered inodes back to the file system
 - removes mount table entry and removes "mount point" mark from the mount point directory

Unmounting File System

- Syntax
 - `umount(special filename);`
 - Special filename : file system to be unmounted
- Algorithm

•Algorithm

```
algorithm unmount
input:  special file name of file system to be unmounted
output: none
{
    if (not super user)
        return(error);
    get inode of special file (algorithm namei);
    extract major, minor number of device being unmounted;
    get mount table entry, based on major, minor number,
        for unmounting file system;
    release inode of special file (algorithm iput);
    remove shared text entries from region table for files
        belonging to file system; /* chap 7xxx */
    update super block, inodes, flush buffers;
    if (files from file system still in use)
        return(error);
    get root inode of mounted file system from mount table;
    lock inode;
    release inode (algorithm iput); /* iget was in mount */
    invoke close routine for special device;
    invalidate buffers in pool from unmounted file system;
    get inode of mount point from mount table;
    lock inode;
    clear flag marking it as mount point;
    release inode (algorithm iput); /* iget in mount */
    free buffer used for super block;
    free mount table slot;
}
```

Figure 5.27. Algorithm for Unmounting a File System

15 Link

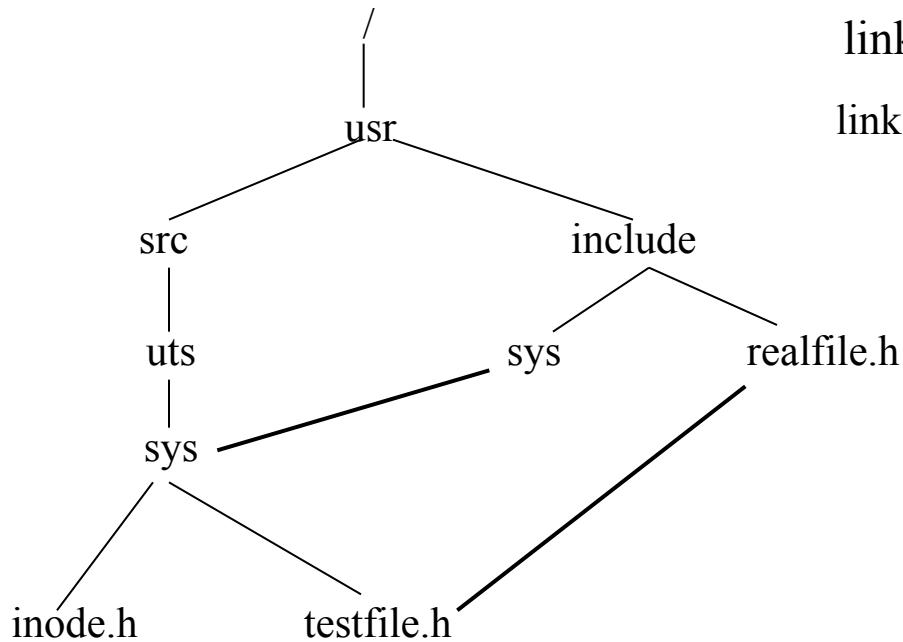
- Syntax

- link(source file name, target file name);
 - source file name : name of an existing file
 - target file name : new name the file will have after completion of the link call

- link

- Links a file to a new name in the file system directory structure, creating a new directory entry for an existing inode

Link Example



`link("/usr/src/uts/sys", "/usr/include/sys")`

`link("/usr/include/realfile.h", "/usr/src/uts/sys/testfile.h")`

Link Algorithm

- Get inode for existing file name
- Increment link count on inode
- Update the disk copy of the inode and unlock the inode
- Search for the target file
 - File is present : link call fail , decrement the link count
 - Otherwise : note the location of empty slot

Algorithm link

Input : existing file name

new file name

Output : none

```
{
    get inode for existing file name (algorithm namei);
    if (too many links on file or linking directory without super user permission)
    {
        release inode (algorithm iput);
        return (error);
    }
    increment link count on inode;
    update disk copy of inode;
    unlock inode;
    get parent inode for directory to contain new file name (algorithm namei);
    if (new file name already exists or existing file, new file on different file systems)
    {
        undo update done above;
        return (error);
    }
    create new directory entry in parent directory of new file name:
        include new file name, inode number of existing file name;
    release parent directory inode (algorithm iput);
    release inode of existing file (algorithm iput);
}
```

16 Unlink

- Syntax
 - `unlink(pathname);`
 - Pathname : name of the file to be unlinked from the directory hierarchy
- `unlink`
 - Removes a directory entry for a file
 - No file is accessible by that name until another directory entry with that name is created
 - `unlink("myfile");`
 - `fd = open("myfile", O_RDONLY);`

Algorithm unlink

Input : file name

Output : none

```
{
    get parent inode of file to be unlinked (algorithm namei);
    /* if unlinking the current directory ... */
    if (last component of file name is ".")
        increment inode reference count;
    else
        get inode of file to be unlinked (algorithm iget);
    if (file is directory but user is not super user)
    {
        release inodes (algorithm iput);
        return (error);
    }
    if (shared text file and link count currently 1)
        remove from region table;
    write parent directory: zero inode number of unlinked file;
    release inode parent directory (algorithm iput);
    decrement file link count;
    release file inode (algorithm iput);
    /* iput checks if link count is 0: if so,
       * releases file blocks (algorithm free) and
       * frees inode (algorithm ifree); */
}
```